

Beyond Alert Chasing: A Wazuh-Based Home Lab for Mastering SOC Workflows and Demonstrating Professional Competence

Strategic Foundations: Designing a Portfolio-Centric Cybersecurity Project

The development of a home lab Security Information and Event Management (SIEM) system represents more than a technical exercise in cybersecurity; it is a strategic endeavor designed to cultivate and validate the precise skills demanded by modern Security Operations Centers (SOCs). The core objective transcends mere tool configuration, aiming instead to replicate the daily workflows of a SOC analyst—log ingestion, query development, alert triage, and anomaly detection—to produce a body of work that serves as a powerful professional portfolio ³. This approach directly addresses the industry's emphasis on hands-on, practical skills over theoretical knowledge alone ³³. By creating tangible artifacts such as custom dashboards, documented alert investigations, and a fully reproducible environment, the project provides concrete evidence of competence to potential employers ¹⁹. The strategic design prioritizes market relevance, realism, and clear communication, ensuring the final output not only demonstrates technical proficiency but also a mature understanding of professional expectations. The explicit exclusion of commercial SIEMs for this foundational project is a deliberate choice rooted in both practicality and career strategy. Free tiers of commercial products often impose significant limitations, such as Splunk's 500 MB per day indexing cap, which would be quickly exhausted in a realistic multi-source lab environment, thereby hindering the ability to practice log analysis at scale ^{33 36}. An open-source stack, conversely, offers unlimited data ingestion, providing the necessary volume to hone the essential skill of distinguishing signal from noise—a task that constitutes approximately 80% of a SOC analyst's daily work ³. Furthermore, building a system from open-source components like Wazuh and the Elastic Stack is highly valued by hiring managers, as it signifies deeper learning, full customization capabilities, and alignment with technologies widely adopted in enterprise environments ^{28 33}.

The selection of Wazuh as the central platform is a cornerstone of this strategy, offering a significant acceleration in achieving a functional, detection-oriented environment. While the raw Elastic Stack provides maximum flexibility, it requires substantial time and effort to configure agents, parsers, and rule sets from scratch [2](#) . Wazuh circumvents this lengthy process by providing a pre-packaged solution that combines SIEM and Extended Detection and Response (XDR) capabilities out of the box [28](#) . It includes pre-built agents for various endpoints, decoders for parsing log files, a robust rules engine, and a comprehensive library of detection rules for common attacks, enabling the creation of a working SOC-like system in days rather than weeks [2](#) [28](#) . This allows the user to immediately focus on higher-value activities such as modifying existing rules, developing custom detections, and writing detailed investigation playbooks. The underlying technology, built upon the Elastic Stack with OpenSearch Dashboards, ensures that the skills learned are transferable to a wide range of modern security tools [28](#) . The choice of Kibana Query Language (KQL) as the primary query language further enhances the project's value proposition. KQL is the standard for Microsoft Sentinel and Defender for Endpoint, making the skills acquired here directly applicable to one of the most popular cloud-native security platforms [4](#) [11](#) . Its intuitive syntax simplifies the process of learning complex querying compared to alternatives like Splunk's Search Processing Language (SPL), allowing for faster proficiency and more effective threat hunting [11](#) [36](#) . This focus on a universal language reinforces the project's goal of building transferable, not just vendor-specific, expertise. The ultimate deliverable is a meticulously structured GitHub repository that functions as a dual-purpose document: a quick-scannable proof-of-concept for time-constrained hiring managers and a detailed, reproducible guide for aspiring peers [19](#) . This structure acknowledges the reality of the recruitment process, where initial screening occurs within seconds, while also demonstrating the ability to communicate complex technical concepts clearly—a crucial skill for collaboration and leadership in any technical role [19](#) [33](#) . The repository's README becomes a masterclass in professional communication, showcasing not just what was built, but how it was built and why it matters, thereby setting the candidate apart from those who simply list certifications on their resume.

The educational philosophy underpinning this project is centered on active learning through simulation and problem-solving. The integration of Atomic Red Team for threat emulation is a critical component that bridges the gap between theory and practice [1](#) . Instead of passively configuring a monitoring system, the user actively tests its defenses against a library of known adversary tactics, techniques, and procedures (TTPs) mapped to the MITRE ATT&CK framework [7](#) [38](#) . This process forces a deep dive into adversary behavior, requiring the user to understand the specific log artifacts an attack generates and then build detection logic around them [16](#) . For instance, simulating a brute-force

attack necessitates understanding login failure patterns in Windows Event Logs or syslog messages on a Linux server and creating a rule that correlates these events over time [2](#) . Each simulated attack and subsequent detection attempt builds a mental model of how attackers operate and how defenders can counter them. This methodology transforms the home lab from a static showcase into a dynamic proving ground, where the user can empirically verify the effectiveness of their detection rules, identify gaps in coverage, and iteratively improve their defensive posture [17](#) . The resulting investigation playbooks become narratives of this adversarial interaction, documenting the analytical journey from a single alert to a comprehensive incident assessment. This process cultivates the "detection engineering mindset," which emphasizes proactive defense-building over reactive alert chasing [17](#) . By focusing on creating custom rules for 10-15 specific MITRE ATT&CK techniques and writing detailed playbooks for 3-5 distinct alerts, the project ensures a deep, rather than superficial, engagement with the material. This depth of experience is far more impressive to a hiring manager than a broad but shallow familiarity with multiple tools. The inclusion of a Raspberry Pi as a sensor adds another layer of realism and complexity, mirroring the diverse and often resource-constrained devices found in real-world networks [20](#) [22](#) . Running lightweight sensors like Suricata or OSSEC on the Pi teaches valuable lessons about network-based intrusion detection and host-based integrity monitoring, expanding the scope of the project beyond traditional log management [25](#) [32](#) . Ultimately, the entire project is designed to answer a fundamental question for any aspiring SOC analyst: "Can you demonstrate your skills?" This project provides a definitive and compelling answer by creating a complete, self-contained, and professionally presented demonstration of capability.

Feature	Rationale for Selection	Impact on Portfolio
Platform: Wazuh (Open Source)	Avoids cost/indexing limits of commercial free tiers 36 , enables deep learning and control, and aligns with widely adopted enterprise technologies 28 33 .	Signals advanced knowledge and practical, cost-effective solutions.
Query Language: KQL	Standard for high-demand tools like Microsoft Sentinel 4 , simpler than SPL 36 , and broadly applicable to modern SIEMs 11 .	Demonstrates forward-thinking and transferable, in-demand skills.
Simulation: Atomic Red Team	Provides safe, standardized tests mapped to MITRE ATT&CK 1 16 , generating realistic log artifacts and structured test cases.	Proves ability to test and validate defenses against known threats.
Deliverables: Playbooks & Docs	Showcases analytical thinking, communication skills, and a mature understanding of SOC workflows 17 19 .	Transcends simple tool use, demonstrating the thought process behind security decisions.
Presentation: Dual-Audience README	Optimizes for recruiter scannability (30-60 seconds) 19 while providing technical depth for peers.	Highlights professionalism and the ability to tailor communication to different stakeholders.

This strategic foundation establishes a clear and actionable path toward building a cybersecurity profile that stands out. It moves beyond the checklist mentality of acquiring skills and focuses on the synthesis of those skills into a cohesive, demonstrable capability. The project is not merely about learning to use Wazuh; it is about learning to think and operate like a professional SOC analyst, armed with a robust set of tools and a clear methodology for defending a network. The resulting portfolio artifact becomes a testament to this transformation, a living document that tells a story of initiative, technical prowess, and professional maturity.

Architectural Blueprint: Selecting and Configuring the Open-Source SIEM Stack

The architectural foundation of this home lab project rests upon a carefully selected open-source stack, prioritizing scalability, deep learning opportunities, and industry alignment. The chosen technology is Wazuh, an open-source platform that integrates Security Information and Event Management (SIEM) and Extended Detection and Response (XDR) functionalities [28](#). Wazuh itself is built upon the Elastic Stack, comprising Elasticsearch for data storage and search, Logstash or Filebeat for data processing and shipping, and OpenSearch Dashboards for visualization [28](#). This layered architecture provides immense power and flexibility. The decision to use Wazuh over a raw Elastic Stack implementation is strategic; it significantly accelerates the setup process by providing pre-configured agents, decoders, and a rich set of detection rules, allowing the user to bypass months of low-level configuration and immediately engage in meaningful tasks like detection engineering and threat hunting [2](#) [28](#). This enables the creation of a functional, detection-oriented environment in a matter of days rather than weeks, maximizing the utility of the project for portfolio development [2](#). The core components of the Wazuh architecture include the Wazuh Server, which hosts the Manager responsible for rule matching and alert generation, and the Wazuh Indexer, which stores and indexes all incoming log data in Elasticsearch [28](#). Communication between the server and clients is managed via the Wazuh API. The entire system can be deployed using Docker containers, which simplifies installation and management, or installed directly on a virtual machine or physical server. For a home lab, a dedicated virtual machine provisioned with sufficient resources (e.g., 4 CPU cores, 8-16 GB RAM, and several hundred gigabytes of disk space) is the recommended approach to ensure stable performance across all components.

The client-side of the architecture involves deploying Wazuh agents on each endpoint whose activity needs to be monitored. These agents collect security-related data from the host system—including file integrity changes, running processes, user activity, and logs—and send it securely to the Wazuh server for analysis. The project requires a diverse set of agent deployments to create a realistic environment. At a minimum, this includes:

1. **Linux Agent:** Deployed on a Linux virtual machine (e.g., Ubuntu Server or CentOS). This agent will be configured to monitor `/var/log/syslog` and other relevant log files, enabling the detection of events such as failed SSH login attempts, software installations, and command-line executions. This directly supports the simulation of techniques like brute-force attacks [2](#).
2. **Windows Agent:** Deployed on a Windows virtual machine (e.g., Windows 10/11 Pro or Windows Server). This agent leverages Windows Event Log collection to ingest events related to process creation, PowerShell execution, account logons, and service installations. This is critical for detecting sophisticated attacks involving PowerShell-based scripts or credential access techniques [14](#).
3. **Raspberry Pi Sensor:** A Raspberry Pi can be configured to run lightweight security tools and forward their logs to the Wazuh server. One option is to install Suricata, an open-source Network Intrusion Detection System (NIDS), which generates `eve.json` logs containing details of detected network events [5](#) [32](#). Another option is to install the OSSEC agent, Wazuh's predecessor, to provide host-based monitoring capabilities [25](#). This addition introduces a non-traditional endpoint and network-level visibility, enhancing the lab's realism [22](#).
4. **Network Device Integration:** Firewalls and routers can be configured to forward their syslog messages to the Wazuh server. This allows for the correlation of security events with network traffic patterns, such as identifying connections to known malicious IP addresses or unusual outbound traffic volumes indicative of data exfiltration.

The overall architecture can be visualized as a star topology, with the central Wazuh/OpenSearch server at the hub and all endpoint agents (Linux VM, Windows VM, Raspberry Pi, etc.) sending data to it. The following table outlines the recommended specifications and roles for each component in the home lab environment.

Component	Recommended Specifications	Role and Configuration Details
Wazuh/ Indexer Server	4+ vCPUs, 16GB+ RAM, 100GB+ Disk	Hosts the Wazuh Manager, API, and Elasticsearch (Indexer). Use Docker Compose for simplified deployment. Configure data retention policies to manage disk usage.
Windows Client VM	2 vCPUs, 4GB RAM, 50GB Disk	Runs Windows 10/11 Pro or Windows Server. Install Wazuh agent. Enable Windows Event Log forwarding for Process Creation, PowerShell, and Logon/Logoff events.
Linux Client VM	2 vCPUs, 4GB RAM, 50GB Disk	Runs Ubuntu Server LTS or similar. Install Wazuh agent. Configure the agent's <code>local_internal_options.conf</code> to tail key log files like <code>/var/log/auth.log</code> and <code>/var/log/syslog</code> .
Raspberry Pi Sensor	Model 4B or newer, 4GB+ RAM, 32GB+ MicroSD	Installs Suricata or OSSEC agent. Suricata requires configuring rulesets (e.g., Emerging Threats Open) and directing <code>eve.json</code> output to a file for the Wazuh agent to monitor.
Network Devices	Virtual Firewall (e.g., pfSense on VM)	Configure syslog forwarding on the device to send logs to the Wazuh server's IP address on UDP port 514.

Once the hardware and virtualization environment are prepared, the next step is the deployment of the Wazuh stack. The official Wazuh documentation provides a comprehensive guide for a production-like setup, which can be adapted for a home lab context [28](#). After installing the Wazuh server and agents, the user must configure the agents to point to the server's IP address and register successfully. This registration process creates entries for each host in the Wazuh dashboard, which can be accessed via a web browser at `https://<wazuh_server_ip>:5601/app/wazuh`. The first-time login credentials are typically provided during the installation script. Post-installation, it is crucial to perform initial hardening, such as changing default passwords, securing the API, and configuring firewall rules to restrict access to the Wazuh web interface and API ports. The configuration of log collection on the agents is a meticulous process. For the Linux agent, this involves editing the `ossec.conf` file to add `<localfile>` sections that specify the path to the target log file and its type (e.g., `syslog`, `auth`). For the Windows agent, configuration is done through the Wazuh manager's web interface or by directly editing the `ossec.conf` file on the agent, where event channels from the Windows Event Log are subscribed to for monitoring. The ability to successfully ingest and parse logs from these varied sources is the first major milestone of the project. This process teaches the fundamentals of log normalization and data structuring, which are essential for effective correlation and analysis in a real SOC. The resulting indexed data forms the corpus upon which all subsequent detection and investigation activities will be built. The successful establishment of this baseline data pipeline is a critical achievement, transforming a collection of disparate log files into a unified security dataset ready for analysis.

Data Ingestion Strategy: Aggregating Multi-Source Log Data for Analysis

A robust data ingestion strategy is the lifeblood of any effective SIEM system, and its successful implementation is the foundational step before any meaningful detection or analysis can occur. The primary goal of this phase is to aggregate security-relevant data from every planned source into the Wazuh index, creating a centralized repository of truth for the home lab environment. This process involves not only getting the data to flow into the system but also ensuring it is correctly parsed, normalized, and enriched with contextual information. The project mandates the ingestion of logs from Windows Event Logs, Linux syslogs, web servers, firewalls, and a Raspberry Pi sensor. Each of these sources presents unique challenges and requires specific configuration. The Wazuh agent, whether on a Linux or Windows host, acts as the primary data collector. On Linux, the agent uses its internal agent options to monitor local log files. The `ossec.conf` file is edited to add `<localfile>` blocks, specifying the absolute path to the log file, its type (`syslog`, `auth`, etc.), and an associated decoder. Decoders are crucial as they contain regular expressions that parse the raw log text and convert it into structured JSON data that the Wazuh server can understand and index. For example, a typical Apache web server log entry like `192.168.1.100 - - [10/Apr/2024:12:00:00 +0000] "GET /index.php HTTP/1.1" 200 1234` can be transformed by a decoder into a structured object containing fields like `client_ip`, `request_method`, `url_path`, `http_response_code`, and `response_size`. This structured format is indispensable for efficient searching and querying with KQL. Similarly, on the Windows client, the Wazuh agent ingests data directly from the Windows Event Log. The agent subscribes to specific event channels identified by their channel name (e.g., `Microsoft-Windows-Sysmon/Operational` for Sysmon events, `Security` for logon/logoff events). Each event is then converted into a Wazuh alert format, making it searchable and correlatable alongside other log types.

To enrich the data and increase its analytical value, several strategies should be employed. First, ensure that all systems are synchronized to a common Network Time Protocol (NTP) server. Accurate and consistent timestamps across all sources are non-negotiable for correlating events chronologically. Second, consider adding metadata to the logs. For example, when configuring log forwarding from a firewall, tag the logs with the zone they originate from (e.g., `'internal'`, `'dmz'`). This can be done by prefixing the log message or by using separate syslog facilities. Third, leverage Wazuh's built-in capabilities for vulnerability detection and configuration auditing. By deploying Wazuh's FIM (File Integrity Monitoring) and vulnerability scanner features, the lab can generate alerts for unauthorized file changes or missing security patches, moving beyond passive log

collection to active compliance monitoring. This provides a richer dataset that reflects not just what happened, but also the state of the system's security posture. The Raspberry Pi sensor adds another dimension to data ingestion. If Suricata is used, its `eve.json` file must be made accessible to the Wazuh agent. The agent can then monitor this file and decode its JSON structure. Each entry in `eve.json` contains detailed information about a network event, including packet headers, payload information, and rule matches. Decoding this requires writing a custom XML decoder for the Wazuh agent to correctly parse the JSON objects. If the OSSEC agent is used on the Pi, it will function similarly to the Linux agent, monitoring system logs and generating alerts for host-based events. This network-centric data is invaluable for detecting lateral movement, C2 beaconing, or data exfiltration attempts that might not be visible from host logs alone.

The culmination of the data ingestion phase is a well-populated Wazuh index containing a diverse and rich set of security data. This data forms the basis for all subsequent activities, particularly the development of detection rules and investigation playbooks. Before proceeding, it is essential to validate the quality of the ingested data. This involves performing exploratory searches in the OpenSearch Dashboards Discover tab to confirm that logs from each source are appearing as expected, with correct timestamps, source identifiers, and parsed fields. Any discrepancies in parsing should be addressed by refining the corresponding decoders. Once the data pipeline is stable and verified, the environment is ready for the next critical stage: threat simulation. This is where the ingested data will be put to the test. By intentionally triggering malicious behaviors, the user can observe the corresponding log entries and begin the process of building detection logic around them. This iterative cycle—simulate an attack, observe the logs, write a detection rule—is the core pedagogical loop of the project. It mirrors the continuous improvement process found in professional SOCs, where defenses are constantly tested and refined. The successful completion of this phase marks a significant achievement, demonstrating a solid grasp of SIEM fundamentals, including log aggregation, parsing, normalization, and data validation. It provides the raw material—the "digital evidence"—necessary to tell compelling stories of detection and investigation, which will form the centerpiece of the final portfolio presentation.

Threat Simulation Framework: Integrating MITRE ATT&CK with Atomic Red Team

With a robust data ingestion pipeline established, the next phase of the project involves actively testing the SIEM's defensive capabilities. This is achieved by implementing a

structured threat simulation framework using Atomic Red Team, an open-source library of small, atomic tests meticulously mapped to the MITRE ATT&CK® adversary tactics and techniques [1](#) [16](#). This approach provides a credible, repeatable, and safe methodology for emulating real-world attacker behavior, moving the project from a passive monitoring system to an interactive defense-testing environment. The primary benefit of this framework is its direct alignment with industry-standard terminology and practices. During security interviews, candidates are frequently asked how they would detect specific ATT&CK techniques, such as T1059 Command and Scripting Interpreter or T1078 Valid Accounts [14](#) [38](#). By using Atomic Red Team, the project directly addresses this expectation, providing verifiable examples of how to detect these exact techniques. Each test in the library corresponds to a specific TTP, complete with detailed documentation on the procedure, expected adversary tools, and potential indicators of compromise (IOCs) [1](#). This structured approach allows for systematic coverage of a wide range of attack vectors, from initial access and execution to persistence and exfiltration.

The implementation process begins with selecting a curated list of 10-12 beginner-to-intermediate level techniques that are likely to generate clear and distinct log artifacts in the Wazuh environment. Based on the available log sources (Windows, Linux, network sensor), a strategic selection would include techniques that exercise the full spectrum of collected data. For example:

MITRE ATT&CK Technique ID	Technique Name	Description	Expected Log Sources
T1078.001	Valid Accounts: Default Accounts	Creates or utilizes accounts with default passwords.	Windows Event Logs (Event ID 4624), Linux Syslogs (/var/log/auth.log)
T1003.001	OS Credential Dumping: NTLM	Attempts to dump NTLM hashes from memory.	Windows Event Logs (Sysmon Event ID 10), Wazuh FIM alerts if registry hives are accessed.
T1021.001	Remote Services: Remote Desktop Protocol	Uses RDP to move laterally within the network.	Windows Event Logs (Event ID 4624, 4625), Firewall Logs (new connection to port 3389), Sysmon Logs (Process Create for mstsc.exe).
T1071.001	Application Layer Protocol: Web Protocols	Exfiltrates data over HTTP/S.	Web Server Logs (unusual request sizes/paths), Firewall Logs (large outbound HTTPS traffic), Sysmon Logs (Process Create for curl/wget).
T1059.001	Command and Scripting Interpreter: PowerShell	Executes commands via PowerShell.	Windows Event Logs (PowerShell transcript logs, Module logging), Sysmon Logs (PowerShell Script Block Logging).
T1021.006	Remote Services: SSH	Brute-forces or gains access via SSH.	Linux Syslogs (/var/log/auth.log for Failed password messages), Wazuh Alerts (for repeated failures).
T1548.002	Account Manipulation: Grant User Right	Grants a user additional privileges (e.g., log on as a service).	Windows Event Logs (Event ID 4704), Wazuh GPO/Configuration Audit alerts.
T1055.012	Process Injection: Thread Execution Insertion	Injects code into a legitimate process thread.	Sysmon Logs (Process Create with parent-child relationships indicating injection).
T1566.002	Phishing: Spearphishing Link	Tricking a user into clicking a link that downloads malware.	Web Server Logs (access to a malicious landing page), Endpoint EDR/AV alerts (if present).
T1048.002	Exfiltration Over Other Network Medium: Ingress to Egress Conversion	Converts data into a different format to bypass filters.	Firewall Logs (unusual protocols/ports), Web Server Logs (data encoded in URL parameters).

After selecting the techniques, the next step is to execute the corresponding Atomic Red Team tests within the home lab environment. This is typically done from a compromised or attacker-controlled machine (which could be another VM or even the Wazuh server itself). Each test script is designed to perform a specific action, such as attempting to log in with a dictionary of passwords (simulating a brute-force attack) or executing a PowerShell command that performs a suspicious action. Following each test, the analyst must investigate the Wazuh dashboard to find the generated alerts. This investigative process is where the core skills are honed. The analyst must search for indicators related to the technique—for example, searching for the username used in a brute-force attempt or the command-line arguments of a suspicious process. This empirical exploration serves two purposes: it validates the effectiveness of existing Wazuh rules and, more importantly, reveals gaps in coverage. If a simulated attack does not trigger an alert, it indicates a deficiency in the detection logic, providing a clear objective for the next phase of the project: detection engineering. This iterative loop of "test -> observe -> learn ->

improve" is the heart of the project and perfectly simulates the operational rhythm of a professional SOC. It demonstrates a proactive, evidence-based approach to security, moving beyond theoretical knowledge to practical application. The results of these simulations, including screenshots of alerts, the KQL queries used to find them, and the reasoning behind them, become the core content for the project's investigation playbooks, providing a rich narrative of defensive success and areas for improvement.

Detection Engineering: Developing Custom Rules and Proactive Hunting Queries

Following the threat simulation phase, the project enters its most analytically demanding stage: detection engineering. This process involves taking the insights gained from observing simulated attacks and translating them into concrete, actionable detection logic. The goal is to create a set of 10-15 custom detection rules that cover the simulated MITRE ATT&CK techniques and supplement the existing Wazuh rule set. This phase embodies the "detection engineering mindset," which focuses on proactively building and refining detection capabilities rather than passively reacting to alerts [17](#). The process begins by analyzing the log data associated with a simulated attack that did not trigger an alert. For example, after simulating T1078 Valid Accounts (Default Accounts), if no alert is generated, the analyst must examine the raw logs in the OpenSearch Dashboards Discover tab. They would look for patterns, such as repeated failed login attempts for a default account name (e.g., 'admin') from a single IP address in the Windows Security log (Event ID 4625). The next step is to translate this pattern into a Wazuh rule. A Wazuh rule consists of several key components: a unique `id`, a `level` (severity), a `description`, and a `<if_sid>` or `<union>` block that defines the conditions for triggering the alert. The condition is typically written using Wazuh's XML-based rule syntax, which can reference fields extracted from the decoded logs. For the brute-force scenario, a new rule might be created that looks for multiple `winlog.event_id="4625"` events with a specific `winlog.event_data.Data.Name` (the username) within a short time window from the same `winlog.event_data.Data.IpAddress`.

The development of these rules requires a deep understanding of both the adversary's TTPs and the structure of the security data. It is not enough to simply match on a keyword; effective rules must correlate events and establish context to reduce false positives. For instance, a rule that triggers on the presence of the word "powershell" in a command line might generate excessive noise from legitimate administrative tasks. A

better rule would correlate a PowerShell process creation event (Sysmon Event ID 1) with specific suspicious parameters, such as `-EncodedCommand` or calls to uncommon modules. The Wazuh documentation provides extensive guidance on crafting effective rules and using decoders to extract the necessary fields [28](#). Once a custom rule is written, it is placed in the appropriate directory on the Wazuh server (e.g., `/var/ossec/etc/rules/local_rules.xml`) and reloaded. The rule is then tested again by re-running the corresponding Atomic Red Team test to verify that it now correctly generates an alert. This iterative process of rule creation, testing, and refinement is central to the project. The final set of 10-15 rules should represent a diverse and thoughtful collection, covering different ATT&CK tactics (Execution, Credential Access, Lateral Movement, etc.) and leveraging different types of log data (process creation, logon events, file access, network connections).

Beyond creating rules for known TTPs, a mature SOC engages in proactive threat hunting. This involves using KQL to search through the vast corpus of historical data for subtle signs of malicious activity that may not have triggered an alert. This practice addresses the "detection gap," where a threat exists in the data but the current logic fails to identify it [18](#). The project should include a dedicated threat hunting section in the documentation, where the user applies KQL to look for anomalies and IOCs. For example, a threat hunter might query for all processes that have spawned from a commonly abused parent process like `svchost.exe` or `explorer.exe`, as this could indicate process hollowing or injection. Another hunt could involve looking for PowerShell commands that exhibit obfuscation techniques, such as the use of character codes or pipeline manipulation. KQL provides powerful tools for this kind of analysis. The `where` clause is used for filtering, `project` for selecting specific columns, `summarize` for aggregating data (e.g., counting logon failures per user), and `bin` for time-binning events to spot spikes in activity. For example, to detect a potential DoS attack, one might use a query like: `SecurityEvents_CL | where EventID == 4625 and Message contains "Logon Type: 3" | summarize FailedLogons=count() by bin(TimeGenerated, 1m), SourceIP_Address_s | where FailedLogons > 100` [4](#). This query would identify IP addresses that generated more than 100 failed logon attempts in any given minute, a strong indicator of a brute-force attack. Documenting these hunting efforts, including the KQL queries used, the findings, and the conclusions drawn, provides another layer of evidence for the portfolio, demonstrating analytical curiosity and a commitment to finding hidden threats. This combination of reactive rule creation and proactive hunting showcases a well-rounded skill set that is highly attractive to employers.

Detection Approach	Objective	Example Technique	KQL/Wazuh Rule Application
Signature-Based Detection	Match known patterns of malicious activity (e.g., command lines, file hashes, IP addresses).	Detecting the use of Mimikatz for credential dumping.	Create a Wazuh rule that triggers on a process command line containing "mimikatz".
Anomaly-Based Detection	Identify deviations from a defined baseline of normal behavior.	Spotting a user logging in from an unusual geographic location.	Use KQL to query authentication logs and flag logins from countries with no business presence.
Correlation-Based Detection	Combine multiple distinct events to infer a more complex attack chain.	Detecting lateral movement via RDP after a successful phishing email.	Create a Wazuh rule that correlates a suspicious email attachment hash with subsequent RDP connection logs.
Threat Hunting	Proactively search historical data for unknown threats and IOCs.	Finding evidence of living-off-the-land binaries (LOLBins) being used abnormally.	Use KQL to find instances of <code>certutil.exe</code> or <code>bitsadmin.exe</code> downloading files from external URLs.

By completing this phase, the home lab evolves from a simple log aggregator into a true defensive system. The custom rules provide a baseline of protection, while the documented threat hunts demonstrate a capacity for deeper analysis. Both deliverables are powerful assets for a portfolio, providing tangible proof of the ability to think critically about security data and engineer effective defenses.

Investigation Playbook Development: From Alert Triage to Incident Response

The culminating phase of the project is the development of detailed investigation playbooks, which serve as the capstone of the portfolio. These documents transform the technical outputs of the lab—alerts, logs, and queries—into compelling narratives of analytical reasoning and incident response. The project requires the creation of 3-5 such playbooks, each detailing the investigation of a specific simulated alert. This process directly mirrors the documentation standards of a professional SOC and provides undeniable evidence of the ability to perform core analyst duties. Each playbook should be structured to tell a complete story, guiding the reader through the entire investigative lifecycle. This not only demonstrates technical skill but also communication ability, a critical attribute for any security professional ¹⁹. The proposed structure for each playbook is as follows: a description of the simulated alert, the KQL query used for the investigation, a summary of the findings, a conclusion regarding the nature of the event, and a final reflection on the lessons learned.

Let's walk through an example for a simulated **T1003.001 - OS Credential Dumping: NTLM** attack.

Playbook Title: Investigating a Simulated NTLM Hash Dumping Attempt

1. The Alert:

- **Description:** A Wazuh alert was triggered indicating a high-severity event related to suspicious process activity. The alert originates from the Windows Client VM. The alert title is "High severity alert from Windows Client." The alert message states: "Suspicious process 'procdump.exe' detected attempting to interact with lsass.exe process. Process command line: 'procdump.exe -ma lsass.exe c:\windows\temp\lsass.dmp'."
- **MITRE ATT&CK Mapping:** T1003.001 - OS Credential Dumping: NTLM.
- **Initial Assessment:** The alert points to a classic credential dumping scenario. `procdump.exe` is a legitimate Sysinternals tool often used by administrators for debugging, but it can also be abused by attackers to dump the memory of the Local Security Authority Subsystem Service (lsass.exe), which contains NTLM password hashes. The alert's high severity is justified by the sensitive nature of the target process and the action being performed.

2. The Investigation Query (KQL): The primary goal is to gather all relevant context surrounding this event. The investigation would start in the OpenSearch Dashboards Discover tab with a query focused on the affected host and time frame of the alert. The initial query might look like this:

```
SecurityEvent_CL
| where Computer == "WINDOWS-CLIENT.DOMAIN.LOCAL"
| and TimeGenerated > ago(1h)
| order by TimeGenerated desc
```

This narrows the field to the last hour of events on the target machine. However, to get a holistic view, a more targeted investigation would involve correlating this event with other data sources. A cross-source KQL query might be constructed to find the user behind the action and any subsequent anomalous behavior:

```
// Find the process tree for the procdump execution
let procdump_process = SecurityEvent_CL
| where EventID == 4688 // Process Creation
| and ProcessName_s == "procdump.exe"
| and CommandLine_s has "lsass.exe"
```

```

| project TimeGenerated, TargetUserName_s, NewProcessName_s, CommandLine_s

// Find the logon event for the user who ran the process
let logon_event = SecurityEvent_CL
| where EventID == 4624 // Logon
| join (procdump_process) on left.TargetUserName==right.TargetUserName_s
| project LogonTime = TimeGenerated, UserName = TargetUserName_s, LogonType = LogonType

// Find any unusual network connections from the host post-event
let suspicious_connections = SecurityEvent_CL
| where EventID == 4624 // Logon
| and TimeGenerated > procdump_process(TimeGenerated) // Assuming a function
| and ProcessName_s in~ ("powershell.exe", "nc.exe", "curl.exe", "wget.exe")
| summarize Connections = count(), LastConnection = max(TimeGenerated) by
    LogonTime, UserName

logon_event
| union (suspicious_connections)
| order by coalesce(LogonTime, LastConnection) desc

```

(Note: The exact KQL syntax for joining subqueries and referencing timestamps may vary and would require refinement during the actual investigation).

3. Findings: The investigation yielded several key pieces of evidence. First, the procdump process was initiated by a user named 'admin_user' who had logged in via a remote desktop session from a workstation in the DMZ subnet (IP 192.168.2.10). This was confirmed by correlating the process creation event with the logon event (Event ID 4624) from the Windows logs. The command line showed the intent to capture the LSASS memory dump. Second, the investigation revealed that 15 minutes after the dump was taken, the same user executed a PowerShell command to compress the newly created .dmp file on the C:\ drive. Third, there were no immediate large-scale outbound connections from the Windows client, suggesting the exfiltration phase had not yet begun. The data formed a coherent timeline: a privileged user logged in remotely, performed a credential dumping action, and then prepared the data for exfiltration.

4. Escalation Decision: Based on the findings, this is not a false positive. The sequence of events—remote login by a privileged user followed by the direct targeting of the LSASS process with a known dumping utility—is a strong indicator of a malicious activity. The preparation to compress the data suggests an imminent data exfiltration attempt. Therefore, the recommendation is to escalate this incident to Tier 2 analysts for immediate containment. Immediate actions should include isolating the 'WINDOWS-

CLIENT' host from the network, revoking the 'admin_user' credentials, and conducting a full forensic analysis of the host.

5. Lessons Learned:

- **Rule Effectiveness:** The Wazuh rule successfully detected the primary malicious action (procdump usage). This demonstrates the value of having specific, signature-based rules.
- **Need for Correlation:** The standalone alert on `procdump` was concerning, but the full picture emerged only after correlating it with logon and subsequent process events. This highlights the importance of building correlation-based rules and the analyst's need to look beyond a single alert.
- **User Context is Key:** The fact that a privileged user performed the action complicates the triage. In a real environment, this might require further verification. However, the combination with the remote logon from an untrusted subnet makes the malicious interpretation far more likely.
- **Future Improvement:** A more advanced detection could be built to specifically look for the `procdump -ma lsass.exe` command pattern, as this is a very specific and easily identifiable TTP. Additionally, a rule could be created to detect the compression of `.dmp` files, signaling the next stage of the attack.

By creating three to five such detailed, multi-faceted playbooks, the user produces a portfolio of work that is not only technically sound but also professionally presented. These documents serve as powerful evidence of analytical maturity, critical thinking, and the ability to communicate complex security issues clearly and concisely. When combined with the architecture diagram, deployment instructions, and a concise executive summary in the main README file, they form a complete and compelling case for the user's readiness to contribute effectively in a real-world SOC environment.

Reference

1. Integrating Wazuh and Atomic Red Teaming <https://www.linkedin.com/pulse/unleashing-security-synergy-integrating-wazuh-atomic-red-chandran-dxdlc>
2. Advanced Threat Detection With Wazuh | PDF <https://www.scribd.com/document/864474099/Advanced-Threat-Detection-With-Wazuh>

3. Ultimate Guide to Building a SOC and SIEM Career in 2025 <https://rokibulroni.com/blog/ultimate-guide-to-soc-siem-career-2025/>
4. How to Detect DoS Attacks in Microsoft Sentinel with KQL https://www.linkedin.com/posts/lalitchoudhary8055_microsoftsentinel-kql-cybersecurity-activity-7360502850646364160_vV2
5. Docs Securityonion Net en 2.3 | PDF | Computer Network <https://www.scribd.com/document/579358411/Docs-Securityonion-Net-en-2-3>
6. Syllabus B.tech CSE 2025 26 WithTrack | PDF <https://www.scribd.com/document/887395024/Syllabus-B-tech-CSE-2025-26-WithTrack>
7. arXiv:2502.10825v1 [cs.CR] 15 Feb 2025 <https://arxiv.org/pdf/2502.10825>
8. Cybersecurity Resilience Demonstration for Wind Energy ... <https://ieeexplore.ieee.org/ielaam/6287639/10005208/10043706-aam.pdf>
9. devops-guidance.pdf <https://docs.aws.amazon.com/pdfs/wellarchitected/latest/devops-guidance/devops-guidance.pdf>
10. Create scheduled analytics rules in Microsoft Sentinel <https://learn.microsoft.com/en-us/azure/sentinel/create-analytics-rules>
11. Advanced Threat Hunting Using KQL | PDF | Security <https://www.scribd.com/document/950891300/Advanced-Threat-Hunting-Using-KQL>
12. Modern Security Data Pipelines: Control Plane for SOC https://www.linkedin.com/posts/software-analyst_soc-cybersecurity-securitydatapipelines-activity-7402710870994427904-Rta_
13. 2025 photonics for agrifood roadmap: towards a ... <https://iopscience.iop.org/article/10.1088/2515-7647/adbea9>
14. Detect script-based threats with ES|QL: MITRE T1059 in ... <https://www.elastic.co/blog/detecting-command-scripting-interpreter>
15. MITRE ATT&CK® coverage | Elastic Docs <https://www.elastic.co/docs/solutions/security/detect-and-alert/mitre-attandckr-coverage>
16. Red Teaming Toolkit: Open-Source Arsenal for Adversary ... https://www.linkedin.com/posts/yildizokan_red-teaming-toolkit-activity-7392112712744439808-o1zj
17. SIEM Detection Engineering Mindset for Modern Security https://www.linkedin.com/posts/zyadelzyat_detectionengineering-cybersecurity-infosec-activity-7417205571252219904-VcV1
18. How AI-Native Data Pipelines Accelerate Threat Hunting https://www.linkedin.com/posts/observo-ai_how-ai-native-data-pipelines-accelerate-threat-activity-7382139844690452480-jown

19. Breaking into Cybersecurity: Practical Advice Needed https://www.linkedin.com/posts/kevinmoore-a9b971a5_i-saw-this-post-im-trying-to-break-into-activity-7413989295180451840-fG-M
20. 利用电信/ICT减少和管理灾害风险 https://www.itu.int/dms_pub/itu-d/opb/stg/D-STG-SG01.03.1-2025-PDF-C.pdf
21. The Abstracts of the 3rd International Online Conference ... <https://www.mdpi.com/2673-9976/54/1/12>
22. Regional perspectives <https://unesdoc.unesco.org/ark:/48223/pf0000385209>
23. Findings of the Association for Computational Linguistics <https://aclanthology.org/volumes/2025.findings-acl/>
24. Hybrid classical-quantum communication networks <https://arxiv.org/html/2502.07298v2>
25. Machine learning environments for cyber-physical systems https://theses.hal.science/tel-05294619v1/file/2023MULH6406_these_RUF.pdf
26. Simple Index <https://mirrors.aliyun.com/pypi/simple/>
27. Website Sitemap | Trainocate <https://trainocate.com/sitemap.html>
28. Corso Intensivo Su Wazuh - Guida Completa | PDF <https://it.scribd.com/document/891688922/Corso-Intensivo-Su-Wazuh-Guida-Completa>
29. ICARST-2025 https://www.iaea.org/sites/default/files/25/04/cn_332_boa.pdf
30. Computer Science <https://arxiv.org/list/cs/new>
31. Advances on Broad-Band Wireless Computing ... <https://link.springer.com/content/pdf/10.1007/978-3-031-76452-3.pdf>
32. IDS IPS SOC Project: Suricata & ELK Stack Implementation https://www.linkedin.com/posts/zainabaarif_cybersecurity-soc-blueteam-activity-7421175723597602816-0442
33. How to Stand Out in Cybersecurity Without Stacking Certs ... https://www.linkedin.com/posts/taimurijlal_how-to-stand-out-in-cybersecurity-without-activity-7297525499327574017-pEb7
34. (PDF) IFEC2023 ProceedingBook https://www.academia.edu/113088156/IFEC2023_ProceedingBook
35. Diccionario Pocket | PDF | Nature <https://www.scribd.com/doc/199669549/Diccionario-Pocket>
36. Shashank Gupta's Post - Splunk Queries for Soc Analyst https://www.linkedin.com/posts/shashank-gupta-745517123_splunk-queries-for-soc-analyst-activity-7388426461541486592-JBE5

37. The Official CompTia Cysa Student Guide Exam Cs0 002 ... <https://www.scribd.com/document/718601781/The-Official-CompTia-Cysa-Student-Guide-Exam-Cs0-002-PDF-Free>
38. MITRE ATT&CK Applications in Cybersecurity and The ... <https://arxiv.org/html/2502.10825v1>
39. (PDF) MITRE ATT&CK Applications in Cybersecurity and ... https://www.researchgate.net/publication/389090450_MITRE_ATTCK_Applications_in_Cybersecurity_and_The_Way_Forward
40. Complete Guide To Cyber Security Operation Center v6.2 ... <https://www.scribd.com/document/902145741/Complete-Guide-to-Cyber-Security-Operation-Center-v6-2-FINAL-1>
41. Free Cyber Resources | PDF | Security <https://www.scribd.com/document/701271299/Free-Cyber-Resources>